

Dokumentation:

Wie haben wir die Aufgabenstellung gelöst?

Zuerst wurde die Aufgabenstellung, in C eine Simulation für ein Parkhaus mit Zeitschritten zu erstellen, analysiert. Diese war grundsätzlich schon in zwei Teile aufgeteilt, was den Anfang erleichterte. Für den ersten Teil bestand die Aufgabe darin, eine strukturierte Planung und ein Grundgerüst für die Parkhaus-Simulation zu entwickeln.

Um zu beginnen, befassten wir uns mit den Vorgaben bezüglich des Abgabebereichs und der Strukturen und erstellten dementsprechend das gewünschte Repository. Im Anschluss wurde der verlangte Abgabebereich analysiert und in Issues im Kanban-Board eingetragen, welche daraufhin gleichmäßig im Team zugewiesen wurden. Die c-Dateien wurden anschließend in Pseudocode erstellt während die h-Dateien direkt implementiert wurden.

Die erarbeitete Lösungslogik ist folgende: Im groben Ablauf unseres Programms kommen Fahrzeuge zufällig und versuchen, in dem Parkhaus zu parken. Dabei wird überprüft, ob das Parkhaus freie Plätze hat. Ist ein freier Platz vorhanden, wird das Fahrzeug direkt eingeparkt und erhält eine zufällig generierte Parkdauer. Sind hingegen alle Stellplätze belegt, wird das Fahrzeug in eine Warteschlange eingereiht. Sobald ein Parkplatz frei wird, wird das nächste Fahrzeug aus der Warteschlange in das Parkhaus übernommen.

Während der Simulation wird für jedes Fahrzeug die verbleibende Parkdauer reduziert. Sobald diese den Wert null erreicht, verlässt das Fahrzeug das Parkhaus, wodurch ein Stellplatz wieder frei wird. Dieser kann anschließend entweder von einem neuen Fahrzeug oder einem wartenden Fahrzeug aus der Warteschlange belegt werden.

Die relevanten Parameter der Simulation, wie beispielsweise die Anzahl der Stellplätze, die maximale Parkdauer oder die Ankunfts Wahrscheinlichkeit, sind zur Laufzeit konfigurierbar. Dadurch kann das Verhalten der Simulation flexibel angepasst und analysiert werden.

Die unterschiedlichen Abläufe werden in mehreren Dateien formuliert, welche in der Main-Datei aufgegriffen werden. Dabei sind die Abläufe in die Dateien parking.c/h, queue.c/h, vehicle.c/h, simulation.c/h und statistics.c/h gegliedert.

Jede Datei übernimmt dabei eine klar definierte Aufgabe. Die Datei „parking“ verwaltet die Stellplätze und enthält die Funktion für das Ein- und Ausparken von Fahrzeugen. Die Datei „queue“ implementiert die Warteschlange vor dem Parkhaus mittels dynamischer Speicherverwaltung. In der Datei „vehicle“ werden die Eigenschaften und die Erstellung von Fahrzeugen definiert. Die Datei „simulation“ steuert den gesamten Ablauf der Simulation und koordiniert die einzelnen Schritte. Schließlich ist das Modul „statistics“ für die Erfassung, Verarbeitung und Ausgabe der Simulationsdaten verantwortlich.

Durch diese Aufteilung konnte eine übersichtliche und erweiterbare Architektur geschaffen werden. Änderungen oder Erweiterungen können so gezielt in einzelnen Modulen vorgenommen werden, ohne das gesamte System zu beeinflussen.

Zusammenfassend wurde die Aufgabenstellung durch eine strukturierte Planung, eine klare Aufteilung in Module sowie die schrittweise Entwicklung der Programmlogik gelöst. Dieses Vorgehen ermöglichte eine solide Grundlage für die anschließende Implementierung im zweiten Teil des Projekts.

Im zweiten Teil der Aufgabenstellung mussten dann die C-Dateien implementiert werden. Dies erfolgte dann auch durch das gleichmäßige Aufteilen. Des Weiteren wurden jeweils 2 implementierte Unit-Tests pro Funktion mit assert.h als auch die Dokumentation verlangt, was im Team wieder aufgeteilt wurde.

Die Aufteilung der Dateien entstand durch das Diskutieren, inwieweit wie viele separate Dateien nötig sind, um die Funktionen zu implementieren. Beispielsweise die Datei „vehicle“ war anfangs Teil von „simulation“, jedoch ergab sich aus den Argumenten, dass eine separate Datei nicht unnötig wäre, da sie weitere Übersichtlichkeit gewährleistet. Des Weiteren wurden verschiedene Lösungswege bezüglich der Wahl der Statistiken diskutiert, da wir uns diese selber überlegen mussten. Neben „Step“ einigten wir uns auch auf „Occupied Spots“ und „Utilization Percent“, die zusammen gute Werte bezüglich der Auslastung liefern, „Queue length“, wodurch der Umgang mit der Ankunftsrate betrachtet werden kann und „Departures/Parked this Step“, die die Anzahl der Autos die im Step das Parkhaus verlassen beziehungsweise parken anzeigen. Bei den Statistiken gab es sehr viele weitere Optionen wie zum Beispiel wie oft es voll besetzt ist, jedoch entschieden wir uns für die genannten, da diese eine sehr informationsreiche Basis bilden, aus denen auch weitere Statistiken entstehen können.

Die Teamarbeit hat sich als erfolgreich erwiesen, da Fragen und Unklarheiten im Plenum schnell beseitigt werden konnten. Die Absprache lief sehr gut, da das Repository von allen regelmäßig überprüft wurde und so beispielsweise Pull Requests schnell überprüft wurden und so auch schnell gutes Feedback erhalten wurde, wodurch das Arbeiten im Allgemeinen sehr effizient ablief. Bei der Aufteilung der Aufgaben legten alle Wert darauf, dass diese möglichst gleichmäßig erfolgt, was sie sehr effektiv machte. Bis auf eine Schulterverletzung gab es keine sonderlichen Schwierigkeiten.